

实验五

实验五：代码优化

姓名：邵与乔 学号：231240019 日期：2026年6月7日

1. 实验目的

- 在实验四生成的中间代码基础上，实现多种**代码优化**技术
- 构建**控制流图（CFG）**和**基本块**，为优化提供分析框架
- 减少冗余指令，提升目标代码质量

2. 实验内容

模块	文件	功能
IR 优化	optimize.c / optimize.h	构建 CFG，执行多遍优化，重构优化后的 IR
中间代码生成	ir.c / ir.h	三地址码生成 + MIPS 输出（同实验四）
其他	复用前四次实验	lexical.l, syntax.y, semantic.c, ast.c 等

3. 实现细节

3.1 控制流图（CFG）构建

- 基本块划分**：以 FUNCTION、标签和跳转指令为边界，将 IR 指令序列划分为基本块
- 边构建**：根据跳转目标和 fall-through 关系建立前驱/后继边
- 循环识别**：通过后向边检测循环，标记循环中的基本块

3.2 优化遍（Pass）流水线

IR → 构建CFG → 局部优化 → 复制传播 → 公共子表达式消除
→ 死分支消除 → 死代码消除 → 循环不变量外提
→ 强度削弱 → 窥孔清理 → 重构IR → 输出

各优化遍说明：

优化遍	功能
局部公共子表达式消除	同一基本块内相同表达式只计算一次，后续引用替换为拷贝
常量折叠	编译时计算常量表达式（如 $\#2 + \#3 \rightarrow \#5$ ）
复制传播	$x := y$ 后，将后续对 x 的引用替换为 y （ y 未被重定义时）
死代码消除	移除其结果不被后续使用的赋值指令
死分支消除	条件跳转的两操作数均为常量时，直接替换为 GOTO 或删除
循环不变量外提	将循环内各次迭代结果不变的表达式移到循环前驱块
强度削弱	将代数简化（ $x * 1 \rightarrow x$, $x + 0 \rightarrow x$, $x * 0 \rightarrow \#0$ ）
窥孔清理	消除自拷贝（ $x := x$ ），合并空基本块

3.3 核心数据结构

// 值表（用于局部 CSE 和常量跟踪）

```
typedef struct {
    char* key;    // 表达式键，如 "v1 + v2"
    char* resultVar; // 持有该值的变量名
    int isConst;
    int constVal;
} ValueEntry;
```

// 指令（解析后的结构化表示）

```
typedef struct {
    IKind kind;
    char dest[64], src1[64], src2[64], op[8], label[64];
    int hasConstVal, constVal;
    int isCopy;
    char copySrc[64];
} Inst;
```

// 基本块

```
typedef struct BasicBlock {
    int id;
    Inst** insts;
    int count;
    struct BasicBlock** preds, **succs;
    int inLoop;
} BasicBlock;
```

4. 代码结构

```
task5/Lab/Code/
├── Makefile
├── lexical.l      # Flex 词法规则
├── syntax.y       # Bison 语法规则 + main 入口
├── ast.h / ast.c  # AST 定义与操作
├── semantic.h / semantic.c # 语义分析
├── ir.h / ir.c    # 中间代码生成 + MIPS 输出
├── optimize.h / optimize.c # IR 优化器[NEW]
└── utils.h
```

编译与运行：

`make`

`./parser ../Test/test1.cmm` # 先语义分析，再生成 IR，再优化，最后输出 MIPS

5. 实验总结

难点

解决方案

CFG 构建中基本块边界判定 函数头、标签、跳转目标均为入口；跳转/返回为出口

优化遍之间的顺序与交互 按照传统编译器优化顺序，最后加多轮清理遍

常量传播的保守性控制 全局常量传播仅作局部实现，避免交汇点的不精确问题

循环不变量检测 检查操作数是否在循环内被定值，若均在循环外则为不变量